DCC207 - Algoritmos 2

Trabalho Prático 1

Indexação Geoespacial e Geometria Computacional Híbrida

Alunos:
Cauã Neto Santos Pires, Rafael Junqueira Baracat, Renato Vilela de Melo

Matrículas: 2024005335, -matricularafael-, 2024066016

ICEx – UFMG
2026/2

Sumário

1 Introdução **2**

2 Coleta de Dados **2**
2.1 Coordenadas dos bares e restaurantes de BH 2
2.1.1 Dataset de Atividade Econômica 2
2.1.2 Requests pela API Geocoder 3
2.2 Grafo da malha viária de BH 3

3 Decisões de Implementação **3**
3.1 Indexação e Busca por Prefixo 4
3.1.1 geohash.py 4
3.1.2 trie.py 5
3.1.3 distance.py 6
3.1.4 Pipeline de Consulta 6
3.2 Simulador de Geração de Entregadores 6
3.2.1 Distribuição Espacial Baseada em Demanda 7
3.2.2 Restrição Viária e Realismo Geográfico 7

4 Decisões de Interface **8**

5 Seção Experimental **8**

6 Conclusões **8**

1 Introdução

Este trabalho tem como objetivo integrar aspectos de implementação da árvore de prefixos (Trie) aplicada à codificação do geohash. Para tal, foi implementado um motor de busca e monitoramento geoespacial híbrido para visualização e consulta de uma frota de entregadores na cidade de Belo Horizonte, de tal modo que possam ser identificados os entregadores mais próximos de um dado restaurante. A implementação foi feita usando somente Python e as estruturas de dados foram feitas com o mínimo de auxílio de bibliotecas externas, usos esses que serão justificados no tópico de decisões de implementação.

2 Coleta de Dados

Nesta seção, detalharemos os procedimentos computacionais adotados para a obtenção, limpeza e tratamento dos dados essenciais ao funcionamento do sistema, divididos entre a geolocalização dos estabelecimentos e a modelagem da malha viária da cidade.

2.1 Coordenadas dos bares e restaurantes de BH

Para o trabalho prático, foi disponibilizada uma base de dados (`bares.csv`) contendo mais de 20 mil estabelecimentos da cidade de Belo Horizonte. Contudo, para a indexação e cálculo de distâncias, era imprescindível associar cada entrada às suas coordenadas geográficas. Esse enriquecimento de dados foi realizado em duas etapas complementares, utilizando recursos abertos da Prefeitura de Belo Horizonte (PBH).

É importante destacar que os dados espaciais fornecidos pela PBH utilizam o sistema de projeção plana SIRGAS 2000 UTM 23S (EPSG:31983). Para assegurar a padronização com o sistema global de posicionamento (WGS 84 - EPSG:4326), adotou-se a biblioteca `pyproj` para a conversão de todas as geometrias obtidas em valores padronizados de latitude e longitude.

2.1.1 Dataset de Atividade Econômica

O primeiro passo de enriquecimento consistiu no cruzamento dos dados originais com o dataset público de Atividades Econômicas da PBH. Através do *script* `merge_bares_csv.py`, os endereços (logradouro e número) de ambas as bases tiveram seus caracteres padronizados nativamente via `Pandas` e foram submetidos a uma operação de *merge* à esquerda. As coordenadas da PBH, fornecidas no formato *Well-Known Text* (WKT), foram extraídas

eficientemente por meio de expressões regulares (*Regex*) e projetadas para WGS 84 de forma vetorizada, garantindo alto desempenho no processamento da base.

2.1.2 Requests pela API Geocoder

Para os estabelecimentos que não encontraram correspondência exata no cruzamento anterior, implementou-se uma rotina de requisições HTTP direcionadas à API Geocoder da PBH (`csv_com_api.py`). O algoritmo itera estritamente sobre os registros pendentes, codifica os endereços em URLs seguras e realiza consultas à API, respeitando um intervalo temporal entre as requisições para mitigar o risco de sobrecarga no servidor ou bloqueio de rede. Por fim, os dados recém-coletados foram acoplados ao arquivo principal por meio do método `combine_first` da biblioteca `Pandas`, que atua preenchendo as lacunas (*NaNs*) do *dataframe* principal com os resultados da API, gerando assim o arquivo consolidado de coordenadas.

2.2 Grafo da malha viária de BH

Para simular a disposição dos entregadores no espaço urbano respeitando as regras de trânsito, fez-se necessário modelar a malha viária de Belo Horizonte. Utilizou-se a ferramenta `OSMnx` (por meio do *script* `get_graph_bh.py`) para extrair um multidigrafo direcionado de vias automobilísticas a partir dos dados do *OpenStreetMap*.

Ademais, para evitar comportamentos anômalos no roteamento — como entregadores presos em quarteirões murados, falhas de mapeamento ou vias exclusivas sem saída —, o *script* `verificar_conexo.py` fez uso da biblioteca `NetworkX` (biblioteca usada pelo `OSMnx` na manipulação de grafos) para filtrar a estrutura. O grafo resultante foi restrito à sua maior componente fortemente conexa.

3 Decisões de Implementação

Nesta seção serão detalhadas as decisões de implementação da indexação e busca por prefixo e do simulador de geração de entregadores.

3.1 Indexação e Busca por Prefixo

3.1.1 geohash.py

Esse arquivo tem como objetivo implementar a conversão de uma coordenada, dada em um par (latitude, longitude), para uma *string* codificada em geohash. A primeira decisão de projeto diz respeito a escolha do alfabeto, que se optou pelo uso padrão sem os caracteres 'a', 'i', 'l' e 'o', adaptado de uma Base32.

Com isso em mente, podemos discutir o porquê da codificação do geohash funcionar. Isso ocorre pela função de conversão se comportar como uma *z-order curve*, ou seja uma função capaz de representar um espaço multidimensional em uma estrutura unidimensional, conservando a noção espacial entre os objetos por meio da intercalação de suas representações em binário.

Logo, explicado isso, podemos partir para a implementação de cada função:

1. **geohash_encode:** começa-se com os intervalos de latitude e longitude comuns do globo, (-90,90) e (-180,180), respectivamente, e passamos a fazer uma busca binária de uma dimensão em cada uma das medidas de coordenadas, ou seja para uma dada média μ verificamos se a opção testada está à direita, cuja resposta codificamos com '1', ou à esquerda, cuja resposta codificamos com '0', da média estabelecida.

   Para mantermos a noção de localidade do geohash, fazemos essa codificação aos turnos, começando por longitude. Após isso, como cada caractere do alfabeto de codificação é de uma Base32, conseguimos codificá-los com 5 bits. Por isso, a cada 5 bits obtidos pela passagem alternada de 0's e 1's geramos um caractere. Ao final, devolvemos a string codificada completa.

2. **geohash_box:** função auxiliar para montarmos o grid necessário para tratar casos em que os entregadores se encontram nas bordas da área definida pela precisão da procura. Para alcançarmos isso, convertemos uma determinada string, já em geohash, para sua representação binária. Depois, por blocos de 5 bits, depois fazemos o caminho inverso do que fizemos na função anterior: lemos os bits e segmentamos os intervalos, visando buscar o centro geométrico da *bounding box* e intervalos de latitude e longitude que a definem.

3. **geohash_grid:** dada a caixa criada na função anterior, podemos gerar uma matriz 3x3 capaz de condensar as proximidades de um ponto central, visando lidar com vizinhanças próximas em consultas que ficam

em branches diferentes da trie, por conta de diferenças na codificação dos prefixos. Assim sendo, essa função permite adquirir as células norte, sul, leste, oeste, nordeste, noroeste, sudeste e sudoeste de uma determinada célula codificada s. Para tal, tomamos o centro geométrico gerado na função anterior e calculamos sua altura h, dada pela diferença entre os delimitadores do intervalo de latitudes também gerado, e seu comprimento w, dado pela diferença entre os delimitadores do intervalo de longitudes igualmente gerado pela função passada. Os pontos cardinais são fruto da adição/subtração de w e h de maneiras diversas nas coordenadas do centro geométrico.

3.1.2 trie.py

Neste arquivo, há a implementação da classe **node_trie**, responsável por codificar um nó da árvore de prefixos, nela há o prefixo que representa sua subárvore, um dicionário de filhos que associa primeiro caractere do prefixo descendente com um objeto *node_trie*. Por fim, há também uma lista de entregadores associados ao nó, que por sua vez são codificados como um dicionário, que armazena seus *id's*, e valores de latitude e longitude.

No mais, há a implementação da classe **compact_trie**, que representa um requisito do trabalho de tornar a estrutura de armazenagem de prefixos eficiente em consulta e em espaço. Nela, há métodos de inserção e busca de prefixo que serão explicados a seguir:

1. **Inserção:** Ao inserir uma chave, determina-se o maior prefixo comum de comprimento k entre a cadeia a ser inserida e o prefixo do nó filho correspondente. Caso k equivalha ao comprimento do prefixo do filho, prossegue-se a descida recursiva com o sufixo restante. Caso k seja estritamente menor, ocorre a operação de *split*: cria-se um nó intermediário rotulado com o prefixo comum de tamanho k, o nó original é encurtado para conter apenas seu sufixo a partir de k e conectado como filho do intermediário, e o que restou da nova chave é alocado em uma nova folha ramificada. Esse mecanismo assegura a invariante da compactação da árvore.

2. **Busca de Prefixo:** A consulta por região recebe um prefixo de tamanho m. O algoritmo navega pelas arestas consumindo fatias correspondentes da chave. Ao identificar o nó onde o prefixo m é integralmente contemplado — etapa que consome tempo proporcional ao tamanho do prefixo, ou seja, $O(m)$ —, dispara-se uma busca em profundidade (DFS) que visita estritamente a subárvore decorrente. A coleta de todos os registros armazenados nas folhas dessa subárvore demanda

tempo proporcional à quantidade k de entregadores recuperados, totalizando a complexidade ótima de $O(m + k)$, independente do número total de nós da árvore.

3.1.3 distance.py

Sabe-se que a aproximação euclidiana do espaço geoide de coordenadas introduz distorções desnecessárias para a implementação. Vide isso, foi adotada a **Fórmula de Haversine**. Considerando o raio médio volumétrico da Terra de $R \approx 6371$km, é calculada a distância no geoide com base nas diferenças angulares em radianos, chamadas Δlat e Δlong:

$$a = \sin^2\left(\frac{\Delta\text{lat}}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\Delta\text{long}}{2}\right)$$

$$d = 2R \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

3.1.4 Pipeline de Consulta

No arquivo **search_engine.py**, é implementado o pipeline de consulta, que importa os métodos e funções previamente descritos.

1. **Seleção Dinâmica de Precisão:** A precisão do geohash de busca deve ser calibrada em função do raio R solicitado na interface. Utilizou-se os seguintes limiares: $R \leq 0.6$km, o que gera a maior precisão de 6; $R \leq 5$km, o que gera uma precisão intermediária para consultas municipais de 5; caso contrário, usamos uma precisão geral 4 para consultas em grandes áreas.

2. **consultar_entregadores:** O motor recebe as coordenadas do restaurante e o raio de atendimento, obtém os 9 prefixos adjacentes e consulta a Trie. Os candidatos recuperados são desduplicados via conjunto de identificadores únicos. Em seguida, calcula-se a distância de Haversine de cada entregador até o restaurante e estima-se o tempo de trânsito considerando uma velocidade média urbana ($v = 25$ km/h). Os registros são segregados entre **filtrados** (distância $\leq R$, ordenados crescentemente para alimentar a interface com os pinos de destaque e tabela) e **descartados** (distância $> R$).

3.2 Simulador de Geração de Entregadores

Para testar o sistema sob condições verossímeis, desenvolveu-se um gerador de dados sintéticos que, ao receber a quantidade N desejada pelo usuário,

cria e posiciona as coordenadas dos entregadores obedecendo a dois princípios matemáticos e geográficos, detalhados a seguir.

3.2.1 Distribuição Espacial Baseada em Demanda

Implementado no arquivo `random_gen.py`, o algoritmo posiciona os entregadores em torno dos estabelecimentos para simular a ociosidade estratégica da frota. Para isso, sorteiam-se N restaurantes como centros, sobre os quais se aplica uma distribuição Gaussiana via `NumPy`.

Para evitar distorções espaciais (visto que linhas de longitude convergem nos polos), a escala do ruído da longitude recebeu uma compensação baseada na secante da latitude média, garantindo uma dispersão estritamente radial:

$$\Delta \text{lon}_{\text{corrigido}} = \Delta \text{lon} \times \left(\frac{1}{\cos(\text{lat}_{\text{média}})} \right)$$

A amplitude desse espalhamento é parametrizada dinamicamente com base na dispersão total dos restaurantes no mapa (`np.ptp`), assegurando coerência com a área urbana.

3.2.2 Restrição Viária e Realismo Geográfico

Para evitar que a distribuição matemática posicione entregadores em locais inacessíveis (como prédios ou corpos d'água), o *script* `coord_correction.py` projeta esses pontos gerados para as vias transitáveis mais próximas.

O algoritmo identifica a aresta viária mais próxima no grafo obtido e filtrado pelo `OSMnx` e extrai sua geometria (curvas originais ou retas modeladas entre nós). Em seguida, a função `nearest_points` — importada da biblioteca `shapely`, que também integra o motor interno do OSMnx — calcula a projeção ortogonal do ponto matemático sobre a linha da rua.

Para otimizar o desempenho dessas operações complexas, que não possuem suporte nativo à paralelização de baixo nível, a função foi encapsulada globalmente com `np.vectorize`. Isso permite processar os N entregadores diretamente como *arrays*, eliminando *loops* ineficientes na execução principal.

4 Decisões de Interface

5 Seção Experimental

6 Conclusões

Referências

[1] Especificação do Trabalho Prático 1 - Indexação Geoespacial e Geometria Computacional Híbrida, DCC/ICEx/UFMG.

[2] Renato Vimieiro. Algoritmos 2, Slides da disciplina.

[3] WIKIPEDIA. *Geohash*. Disponível em: https://en.wikipedia.org/wiki/Geohash. Acesso em: 17 out. 2026.

[4] WIKIPEDIA. *Z-order curve*. Disponível em: https://en.wikipedia.org/wiki/Z-order_curve. Acesso em: 17 out. 2026.

[5] WIKIPEDIA. *Haversine formula*. Disponível em: https://en.wikipedia.org/wiki/Haversine_formula. Acesso em: 17 out. 2026.